

TP4 - Forêt d'arbres aléatoires

I. Chargement et visualisation des données

Chargez les données (TP5a.npy) et visualisez-les avec la commande :

```
data = np.load("TP4.npz")
X_train, y_train, X_test, y_test = (data[key] for key in ["X_train", "y_train", "X_test", "y_test"])
plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train, s=1, cmap='rainbow');
plt.show()
```

Questions

Combien y a-t-il de points dans la base d'apprentissage ? Dans la base de test ? Quelle est la dimension des données ?

II. Arbre de décision

a. Principe des arbres de décision

On réalise la classification avec un arbre de décision (fonction `visualize_classifier`). Pour reproductibilité lors de la correction, la variable `random_state` correspondra aux trois derniers chiffres de votre numéro d'étudiant.

```
tree1 = DecisionTreeClassifier(criterion='entropy', max_depth = 3, random_state= ****)
tree1.fit(X_train, y_train)
visualize_classifier(tree1, X_train, y_train)

tree.plot_tree(tree1)
plt.show()
text_representation = tree.export_text(tree1)
print(text_representation)
```

Questions

Que représente la variable `max_depth` ?

Qu'observe-t-on avec `visualize_classifier`, `tree.plot_tree` et `tree.export_text` ?

Retrouvez-vous visuellement toutes les découpes sur la figure issue de `visualize_classifier` ?

b. Performance d'un classifieur multi-classes

Réalisez la classification sur la base de test avec :

```
y_pred = tree.predict(X_test)
C=confusion_matrix(y_test, y_pred)
print(classification_report(y_test, y_pred))
print('Accuracy= ',accuracy_score(y_test, y_pred))
```

Questions

Que représentent les valeurs renvoyées par `classification_report` ?

Retrouvez par le calcul la première ligne (classe 1) renvoyée par `classification_report` à partir de la matrice de confusion.

c. Optimisation de la profondeur de l'arbre

Faites varier `max_depth` et estimez pour chaque valeur le taux de reconnaissance. Conclusion sur l'évolution du taux de reconnaissance en fonction de la profondeur de l'arbre.

Affichez le partitionnement de l'espace obtenu pour l'arbre avec le meilleur taux de classification

Questions

Est-ce que ce partitionnement vous paraît visuellement satisfaisant ? Commentez.

d. Arbre de décision sur des données de grande dimension

Chargez maintenant les données TP4b.npz et utilisez la question précédente pour optimiser la profondeur de l'arbre.

Questions

Combien y a-t-il de points dans la base d'apprentissage ? Dans la base de test ? Quelle est la dimension des données ?

Quel est le taux de classification obtenu avec le meilleur classifieur ?

III. Forêt d'arbres aléatoires**a. Test d'une forêt d'arbres aléatoires**

Utilisez RandomForestClassifier pour construire une forêt d'arbres aléatoires. Pour reproductibilité lors de la correction, la variable random_state correspondra aux trois derniers chiffres de votre numéro d'étudiant.

```
from sklearn.ensemble import RandomForestClassifier
RF = RandomForestClassifier(criterion='entropy', n_estimators=10, random_state=***)
RF.fit(X_train, y_train)
```

Testez le classifieur avec les paramètres par défaut et 10 arbres. Conclusion.

b. Influence des paramètres

- Observez l'évolution du taux de bonne classification en fonction du nombre d'arbres. Conclusion sur le meilleur compromis temps/Taux.
- En conservant le nombre d'arbres optimal, observez l'évolution du taux de bonne classification en fonction de la profondeur de l'arbre. Conclusion.
- En conservant le nombre d'arbres et la profondeur, observez l'évolution du taux de bonne classification en fonction du nombre de caractéristiques utilisées lors du bagging. Conclusion.

Est-ce que toutes ces évolutions étaient prévisibles ?

c. Choix des paramètres optimaux

Utilisez la fonction GridSearchCV() pour optimiser les paramètres de la forêt autour des valeurs trouvées précédemment.

Quel est le taux de reconnaissance obtenu avec les paramètres trouvés par GridSearchCV ?

Conclusion

Questions

Que représente le paramètre cv de GridSearchCV ?